

7	Design and implement a menu driven program in C for the following operations on Doubly Linked List (DLL) of Student Data with the fields: USN, Name, Department, Marks, Ph.No, a. Create a DLL of N Students Data by using end insertion. b. Display the status of DLL and count the number of nodes in it c. Perform Insertion and Deletion at End of DLL
	d. Perform Insertion and Deletion at Front of DLL e. Display the total and average marks for each student

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct Student {
    char usn[20];
    char name[50];
    char dept[30];
    int marks;
    char phone[20];
    struct Student *prev, *next;
};
```

```
struct Student *head = NULL;
```

```
// Create a new node
```

```
struct Student* createNode() {
    struct Student *newnode = (struct Student*)malloc(sizeof(struct Student));
    printf("Enter USN: ");
    scanf("%s", newnode->usn);
    printf("Enter Name: ");
    scanf("%s", newnode->name);
    printf("Enter Dept: ");
    scanf("%s", newnode->dept);
    printf("Enter Marks: ");
    scanf("%d", &newnode->marks);
    printf("Enter Phone: ");
    scanf("%s", newnode->phone);
    newnode->prev = newnode->next = NULL;
    return newnode;
}
```

```
// a. Create DLL using end insertion
```

```
void createDLL(int n) {
    struct Student *temp;
    for (int i = 0; i < n; i++) {
        struct Student *newnode = createNode();
        if (head == NULL) {
            head = newnode;
        } else {
            temp = head;
            while (temp->next != NULL)
                temp = temp->next;
            temp->next = newnode;
            newnode->prev = temp;
        }
    }
}
```

```

    }
}
}

```

// b. Display DLL + count nodes

```

void displayDLL() {
    if (head == NULL) {
        printf("\nDLL is empty!\n");
        return;
    }
    struct Student *temp = head;
    int count = 0;
    printf("\nDLL (Horizontal Display):\n");
    while (temp != NULL) {
        printf("[ %s | %s | %s | %d | %s ]",
            temp->usn, temp->name, temp->dept,
            temp->marks, temp->phone);
        if (temp->next != NULL)
            printf(" <-> ");
        temp = temp->next;
        count++;
    }
    printf("\nTotal Nodes: %d\n", count);
}

```

// c. Insert at End

```

void insertEnd() {
    struct Student *newnode = createNode();
    if (head == NULL) {
        head = newnode;
        return;
    }
    struct Student *temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newnode;
    newnode->prev = temp;
}

```

// c. Delete at End

```

void deleteEnd() {
    if (head == NULL) {
        printf("\nDLL is empty!\n");
        return;
    }

```

```

    struct Student *temp = head;

```

// Single node

```

if (temp->next == NULL) {
    printf("Deleting: %s\n", temp->name);
    head = NULL;
    free(temp);
    return;
}

```

```

    while (temp->next != NULL)
        temp = temp->next;
    printf("Deleting: %s\n", temp->name);
    temp->prev->next = NULL;
    free(temp);
}

// d. Insert at Front
void insertFront() {
    struct Student *newnode = createNode();
    if (head == NULL) {
        head = newnode;
        return;
    }
    newnode->next = head;
    head->prev = newnode;
    head = newnode;
}

// d. Delete at Front
void deleteFront() {
    if (head == NULL) {
        printf("\nDLL is empty!\n");
        return;
    }
    struct Student *temp = head;
    // Single node
    if (temp->next == NULL) {
        printf("Deleting: %s\n", temp->name);
        head = NULL;
        free(temp);
        return;
    }
    printf("Deleting: %s\n", temp->name);
    head = head->next;
    head->prev = NULL;
    free(temp);
}

// e. Display total + average marks
void showMarks() {
    if (head == NULL) {
        printf("\nDLL is empty!\n");
        return;
    }
    struct Student *temp = head;
    int total = 0, count = 0;
    while (temp != NULL) {
        total += temp->marks;
        count++;
        temp = temp->next;
    }

    float avg = (float)total / count;
    printf("\nTotal Marks of all Students: %d\n", total);
    printf("Average Marks: %.2f\n", avg);
}

```

```

int main() {
    int choice, n;
    while (1) {
        printf("\n===== DOUBLY LINKED LIST MENU =====\n");
        printf("1. Create DLL using End Insertion\n");
        printf("2. Display DLL & Count Nodes\n");
        printf("3. Insert at End\n");
        printf("4. Delete at End\n");
        printf("5. Insert at Front\n");
        printf("6. Delete at Front\n");
        printf("7. Display Total & Average Marks\n");
        printf("8. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("How many students? ");
                scanf("%d", &n);
                createDLL(n);
                break;
            case 2:
                displayDLL();
                break;
            case 3:
                insertEnd();
                break;
            case 4:
                deleteEnd();
                break;
            case 5:
                insertFront();
                break;
            case 6:
                deleteFront();
                break;
            case 7:
                showMarks();
                break;
            case 8:
                return 0;
            default:
                printf("Invalid choice!\n");
        }
    }
}

```